

Interaktywny Symulator Rdzenia i Pulpitu Operatorskiego Reaktora Jądrowego z Warstwą Predykcji Anomalii

Autorzy: Barbara Składanowska • Katarzyna Tritchart • Tycjan Harczak

1. WSTĘP I CEL PROJEKTU

Tło projektowe

Projekt przedstawia zintegrowany system symulacji fizyko-termicznej rdzenia reaktora jądrowego, strukturalnie inspirowanego reaktorami kanałowymi dużej mocy RBMK-1000.

Główne założenia systemu:

- Numeryczne modelowanie fizyki reaktora i sprzężeń zwrotnych ($\Delta t = 1.0$ s).
- Projekt ergonomicznego interfejsu HMI w Grafanie.
- Asynchroniczna komunikacja i ochrona przed zakleszczeniami (FastAPI, Redis).
- Detekcja anomalii stanów przejściowych przez Autoencoder LSTM.

2. ARCHITEKTURA SYSTEMOWA

Przepływ danych w sieci Docker (Event-Driven Architecture)

- **Warstwa API (api.py):** FastAPI obsługuje punkty końcowe (/prety, /water, /in_water_temp, /az5) z walidacją Pydantic i middleware CORS.
- **Broker danych (redis_client.py):** Asynchroniczne buforowanie komend operatorskich w pamięci in-memory za pomocą operacji LPUSH.
- **Pętla główna (app.py, database.py):** Taktowanie dyskretne co 1.0 s, pobieranie komend operacją RPOP (kolejka FIFO) i synchroniczny zapis stanu (SYNCHRONOUS) do bazy InfluxDB.

```
// Pipeline architektury:  
[Grafana Dashboard]  
  | (HTTP POST Webhooks)  
  ▼  
[FastAPI Server]  
  | (LPUSH)  
  ▼  
[Redis Cache]  
  | (RPOP)  
  ▼  
[Python Engine] —(Sync Write)—> [InfluxDB]  
  | (Query 30m)  
  ▼  
[PyTorch LSTM Inference] —————> [InfluxDB]
```

3. MODEL FIZYKI I TERMOHYDRAULIKI

Bilans cieplny i parowanie

Przyrost temperatury i generacja fazy parowej (v_{steam}) przy przekroczeniu progu wrzenia 284.0°C :

$$\Delta T = k_{\text{termo}} * (P_{\text{thermal}} / Q_{\text{coolant}})$$
$$T_{\text{outlet}} = T_{\text{inlet}} + \Delta T$$

$$V_{\text{steam}} = (3.068 * (P_{\text{thermal}} / Q_{\text{coolant}})) - (M_{\text{subcooling}} * 0.005)$$

Bilans reaktywności ($\Delta\rho$) i moc

$$\Delta\rho = \rho_{\text{base}} + \rho_{\text{void}} + \rho_{\text{xenon}} + \rho_{\text{rods}}$$

- **ρ_{void}** : Efekt pustynny ($V_{\text{steam}} * 4.5$).
- **ρ_{xenon}** : Zatrucie ksenonowe ($-4.0 * \text{xenon_level}$).
- **ρ_{rods}** : Układ regulacji ($-(\text{ORM} * 0.06) + (P_{\text{inserted}} * 0.02)$).

Wykładnicza zmiana mocy cieplnej zależy od całkowitej reaktywności i stałej czasowej reaktora (τ).

4. INTERFEJS HMI I LOGIKA FLUX

Dychotomiczny podział prezentacji prętów kontrolnych (0 - 211 sztuk)

W celu wyeliminowania błędów interpretacyjnych operatora zrezygnowano ze wskaźnika procentowego na rzecz dwóch paneli Bar Gauge:

Partially Inserted Rods (Pcs)

Liczba prętów w reżimie automatyki (kolor zielony).

```
from(bucket: "reactor_test")
|> filter(fn: (r) => r._field == "partially_inserted") |> last()
|> map(fn: (r) => ({ r with _value: math.round(x: (float(v: r._value) /
100.0) * 211.0) })))
```

Fully Inserted / SCRAM (Pcs)

Pręty w dolnym położeniu krańcowym (zrzut ochronny).

```
from(bucket: "reactor_test") |> last()
|> pivot(rowKey:["_time"], columnKey: ["_field"], valueColumn: "_value")
|> map(fn: (r) => { partial_pcs = math.round(x: (float(v:
r.partially_inserted) / 100.0) * 211.0)
return { _time: r._time, value: float(v: r.orm_value) - partial_pcs } })
```

5. WARSTWA GŁĘBOKIEGO UCZENIA (AI)

Przygotowanie danych (dataset.py)

- Ładowanie binarnego formatu Apache Parquet.
- **Okno przesuwne (Sliding Windows):** Macierz danych [50, 12] odpowiadająca 50 sekundom ciągłego monitoringu 12 sensorów.
- Normalizacja statystyczna centrująca średnią wokół 0 i sprowadzająca odchylenie standardowe do 1.

Architektura LSTM Autoencoder (model.py)

- Paradygmat uczenia nienadzorowanego (Unsupervised Learning).
- **Encoder (Koder):** 2 warstwy LSTM (hidden_size=64, dropout=0.2). Kompresja sekwencji czasowej do przestrzeni ukrytej (Bottleneck).
- **Decoder (Dekoder) & Warstwa Liniowa:** Odtworzenie pierwotnej sekwencji [50, 12] i mapowanie na oryginalną przestrzeń cech.

6. PROCES UCZENIA I KLASYFIKACJA

Trening modelu (train.py)

- Trening wyłącznie na danych reprezentujących normalną i bezpieczną pracę reaktora.
- **Funkcja straty:** Błąd średniokwadratowy (MSE) penalizujący duże błędy kwadratowo.
- **Mechanizmy pętli:** Gradient Clipping (do 1.0), Early Stopping (patience=10) oraz optimizer Adam z ReduceLROnPlateau.

Klasyfikacja i Próg (model_to_db.py)

- Wyznaczanie progu statystycznego (Threshold) przy użyciu **95. percentyla błędów rekonstrukcji** zbioru walidacyjnego:

```
Threshold = P95(MSE_val)
```

- W przypadku nieliniowej awarii sieć nie potrafi poprawnie odtworzyć sygnału, błąd MSE gwałtownie rośnie. Jeśli MSE > Threshold -> zapis statusu **is_anomaly = 1** do InfluxDB.

7. KONTENERYZACJA I SCENARIUSZ TESTOWY

Środowisko Docker (`docker-compose.yml`)

- **pytorch-model-dev:** Kontener aplikacyjny PyTorch ze wsparciem GPU NVIDIA CUDA (driver: nvidia) dający 10–100× przyspieszenie uczenia. Wykorzystuje Docker Volumes do mapowania kodu i plików Parquet.
- **influxdb-dev:** Izolowany kontener bazy danych InfluxDB (wersja 2.7–alpine) na wolumenie influxdb-data.

Wyniki 3-fazowego testu dynamicznego

1. **Krytyczność nominalna:** Stabilne 3200 MW, $\Delta p = 0.0$, minimalny błąd MSE = 0.005.
2. **Kryzys termohydrauliczny:** Spadek przepływu chłodziwa poniżej 30000 m³/h, parowanie, wzrost mocy. Sieć wykrywa odchyłkę: MSE > Threshold, alarm w Grafanie.
3. **Procedura AZ-5:** Zrzut awaryjny 211 prętów, reaktywność spada poniżej zera, moc wygasza się do poziomu ciepła powyłączeniowego (160 MW).

7. Grafana dashboard



8. Podsumowanie i Wnioski

- **Pełna integracja systemowa (1 Hz):** Połączenie bazy InfluxDB, brokera Redis oraz serwera FastAPI zapewniło całkowitą stabilność przesyłu metryk i bezopóźnieniową obsługę komend operatorskich.
- **Brak produkcyjnego wdrożenia AI:** W toku prac projektowych nie zaimplementowano ostatecznie działającego modelu predykcji anomalii online; opracowany potok danych oraz struktura Autoencodera LSTM w PyTorch zostały pozostawione jako fundament pod dalszy rozwój systemu.
- **Realizacja standardu HMI:** Pomimo braku aktywnej warstwy AI, zaprojektowany ekosystem w obecnej konfiguracji w pełni realizuje wszystkie pozostałe założenia funkcjonalne, tworząc bezpieczny i spójny pulpit operatorski klasy przemysłowej

10. OŚWIADCZENIE O WYKORZYSTANIU TECHNOLOGII ASYSTUJĄCYCH

W procesie redakcji, korekty językowej oraz ustrukturyzowania dokumentacji technicznej wykorzystano model językowy Gemini (Google). W celu optymalizacji i weryfikacji warstwy analitycznej modeli uczenia maszynowego wykorzystano pakiety Cloud Somnet 4.6 oraz HQ 4.5. Logika i kod źródłowy systemu są wynikiem samodzielnej pracy autorów.